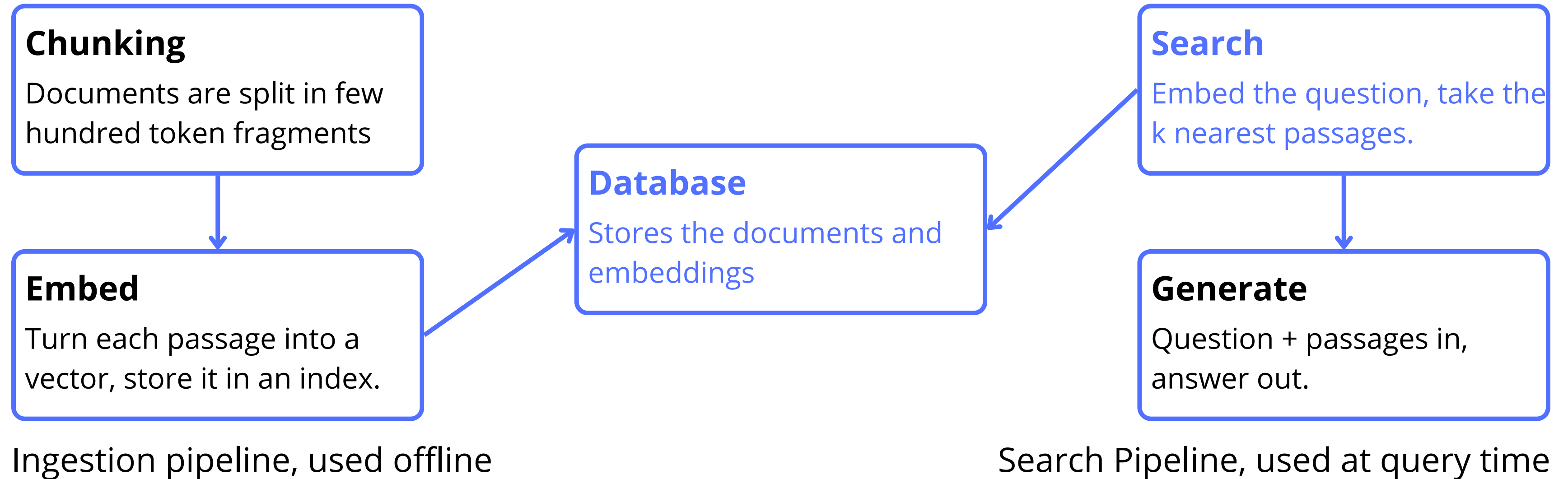


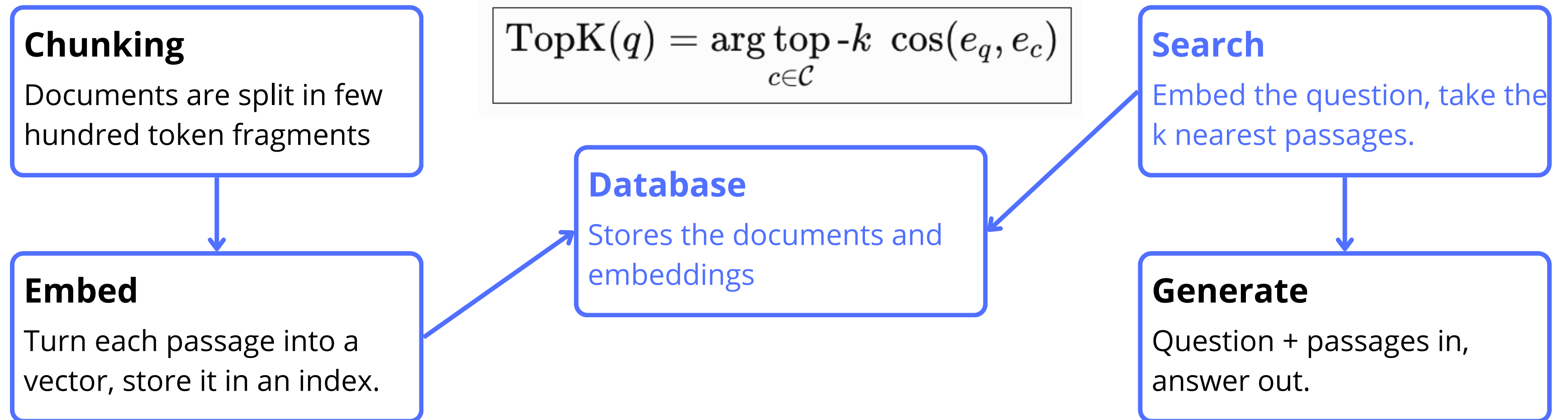
Agentic and Graph RAG

Adrian Valica

Retrieval Augmented Generation



Retrieval Augmented Generation



Ingestion pipeline, used offline

Search Pipeline, used at query time

We look at the meaning of the query, and we retrieve the chunks that are closest in terms of meaning to those chunks.

Retrieval Augmented Generation



Ingestion pipeline, used offline

Search Pipeline, used at query time

We look at the meaning of the query, and we retrieve the chunks that are closest in terms of meaning to those chunks.

Problem for precise queries → Hybrid Search

Retrieval Augmented Generation

This RAG functioning assumes that the query has a meaning, and that the answer will be based on the documents that have the closet meanings.

Qe vs Qc

Similarity = Relevance

“How does our refund policy compare to the old one, which is stricter, and who abused it the most?”

Retrieval Augmented Generation

This RAG functioning assumes that the query has a meaning, and that the answer will be based on the documents that have the closet meanings.

Qe vs Qc

Similarity = Relevance

“How does our refund policy compare to the old one, which is stricter, and who abused it the most?”

Multiple Objectives, average meaning is unclear.

“Who runs the team that owns the service behind last month's most critical outage?”

Retrieval Augmented Generation

This RAG functioning assumes that the query has a meaning, and that the answer will be based on the documents that have the closet meanings.

Qe vs Qc

Similarity = Relevance

“How does our refund policy compare to the old one, which is stricter, and who abused it the most?”

Multiple Objectives, average meaning is unclear.

“Who runs the team that owns the service behind last month's most critical outage?”

Answer is built from multiple intermediary documents.

“What other critical consequences were caused by the disconnection?”

Retrieval Augmented Generation

This RAG functioning assumes that the query has a meaning, and that the answer will be based on the documents that have the closet meanings.

Qe vs Qc

Similarity = Relevance

“How does our refund policy compare to the old one, which is stricter, and who abused it the most?”

Multiple Objectives, average meaning is unclear.

“Who runs the team that owns the service behind last month's most critical outage?”

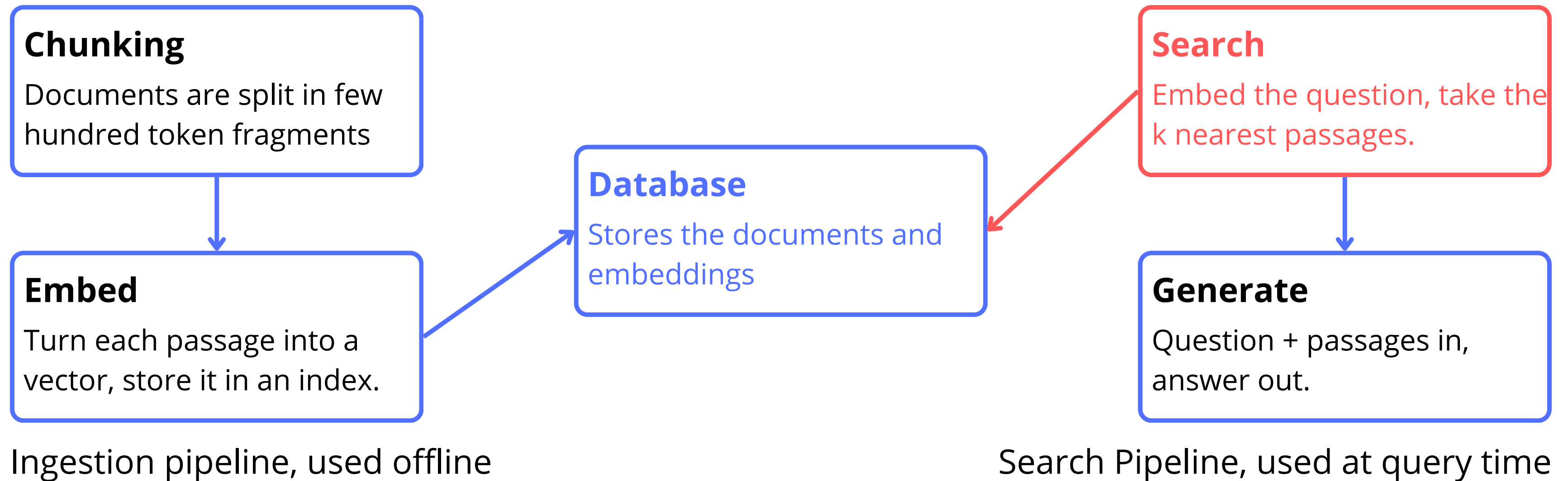
Answer is built from multiple intermediary documents.

“What other critical consequences were caused by the disconnection?”

The question is about the corpus, not a specific document.

Retrieval Augmented Generation

“How does our refund policy compare to the old one, which is stricter, and who abused it the most?” **Multiple Objectives, average meaning is unclear.**

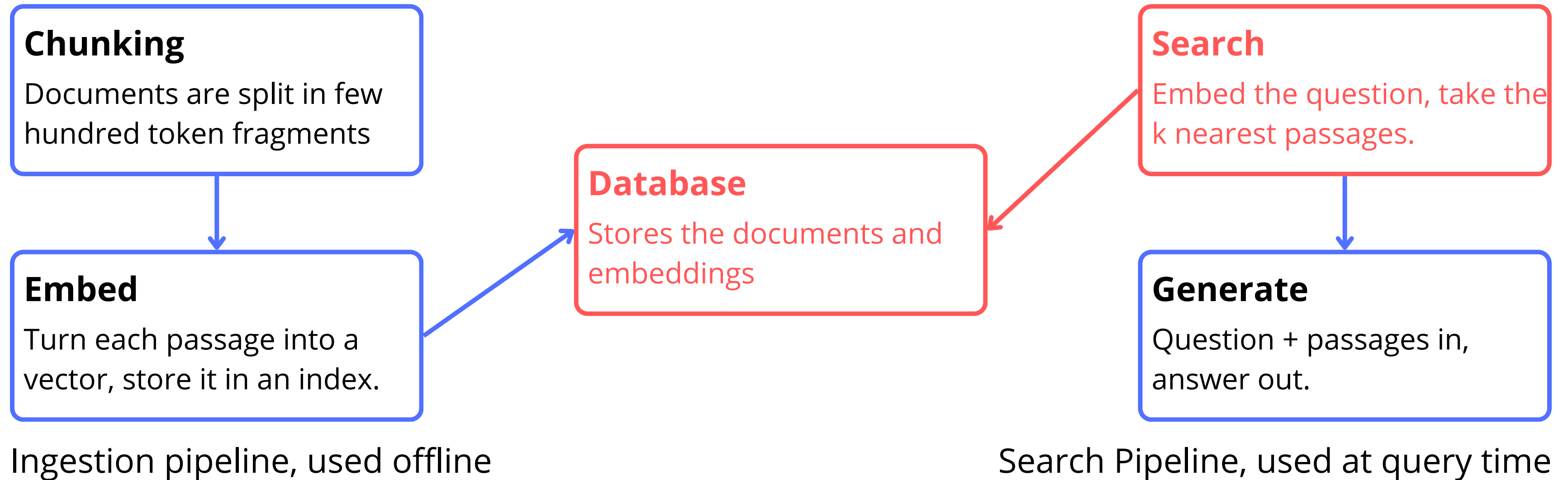


We could find a way to search based on each objective of the query.

Retrieval Augmented Generation

“Who runs the team that owns the service behind last month's most critical outage?”

Answer is built from multiple intermediary documents.

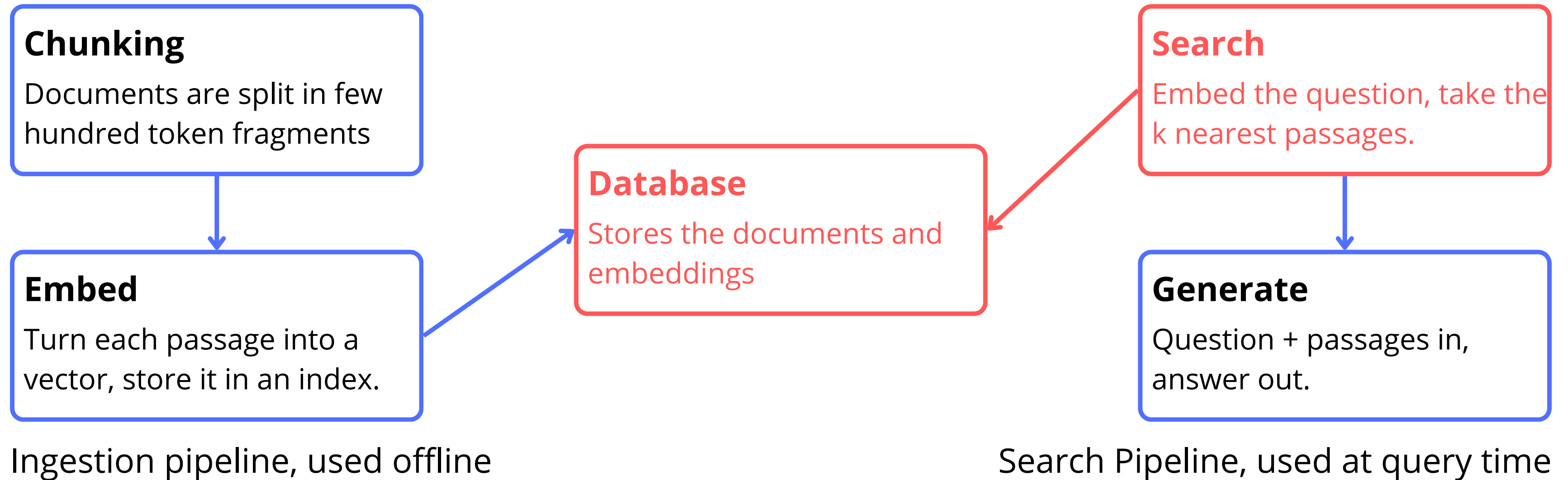


We could find a way to have some kind of “relationships” in the corpus.

Retrieval Augmented Generation

“What other critical consequences were caused by the disconnection?”

The question is about the corpus, not a specific document.

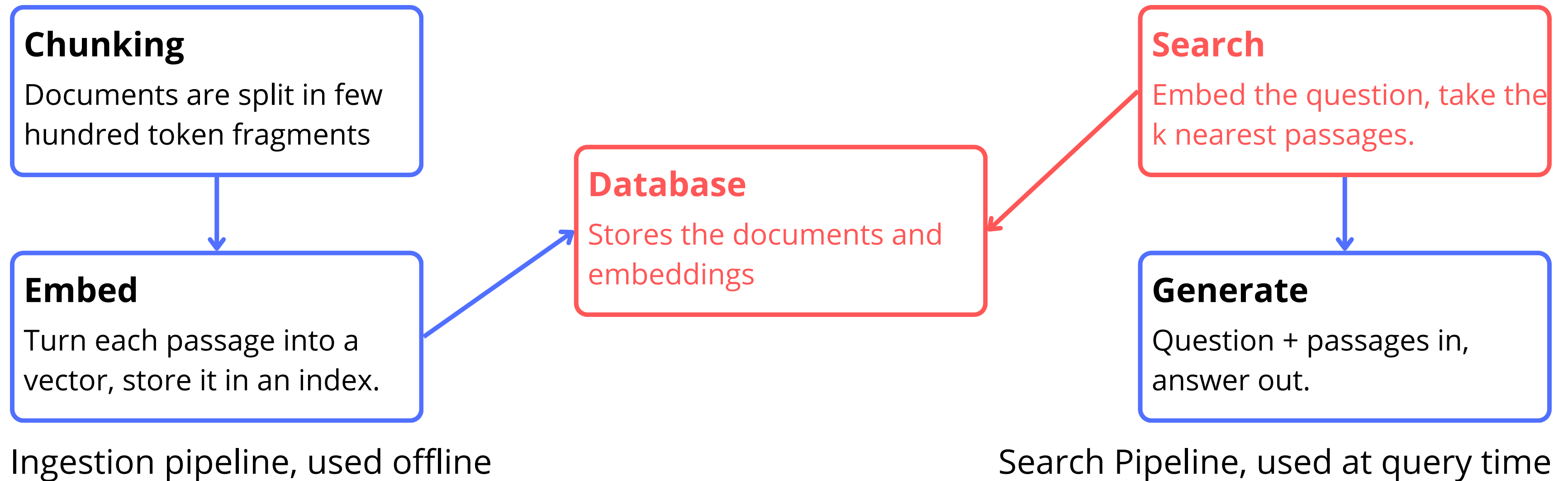


We could have a way to traverse the corpus from a certain point.

Retrieval Augmented Generation

“What other critical consequences were caused by the disconnection?”

The question is about the corpus, not a specific document.



We could have a way to traverse the corpus from a certain point.

This systemic changes should be Query and Corpus independent, dynamic.

Agentic RAG

First, let's fix the mixed up, unclear queries.

"How does our refund policy compare to the old one, which is stricter, and who abused it the most?"

Unclear Query  **Rephraser**  **A set of clear objectives**

Agentic RAG

First, let's fix the mixed up, unclear queries.

"How does our refund policy compare to the old one, which is stricter, and who abused it the most?"

Unclear Query  **Rephraser**  **A set of clear objectives**

"How does our refund policy compare to the old one?"

"Which one of the new policy and old policy is stricter?"

"Who abused the old policy the most?"

Agentic RAG

First, let's fix the mixed up, unclear queries.

"How does our refund policy compare to the old one, which is stricter, and who abused it the most?"

Unclear Query → **Rephraser** → **A set of clear objectives**

"How does our refund policy compare to the old one?"
"Which one of the new policy and old policy is stricter?"
"Who abused the old policy the most?"

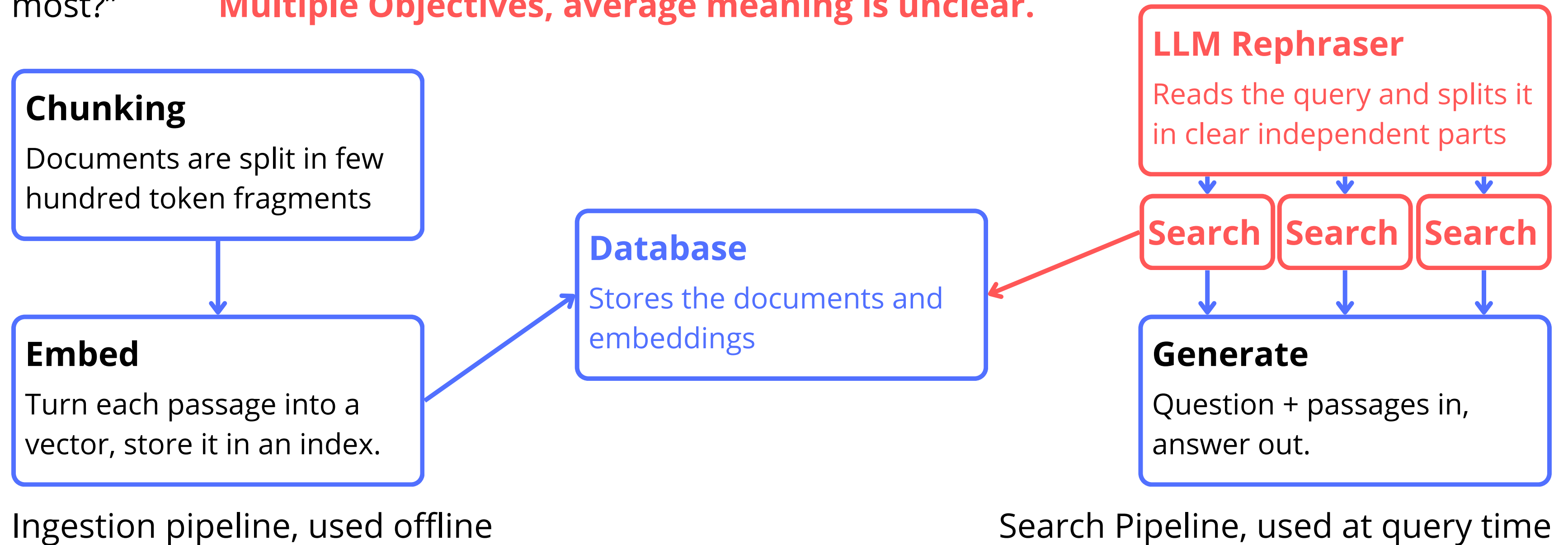
LLM Rephraser

Reads the query and splits it
in clear independent parts

The classical RAG can then run a search for each one of these subqueries.
All the retrieved document are given to the LLM as evidence to answer the query.

Agentic RAG

“How does our refund policy compare to the old one, which is stricter, and who abused it the most?” **Multiple Objectives, average meaning is unclear.**



We could find a way to search based on each objective of the query.

Why Agentic?

In a classical RAG pipeline, the model only intervenes at the end.

Here, the model is used before hand to take actions that will help in solving the query.

Its answers don't give a result, they change the flow of actions that come next.

Unclear Query —————> **Rephraser** —————> **A set of clear objectives**

“How does our refund policy compare to the old one?”
“Which one of the new policy and old policy is stricter?”
“Who abused the old policy the most?”

LLM Rephraser

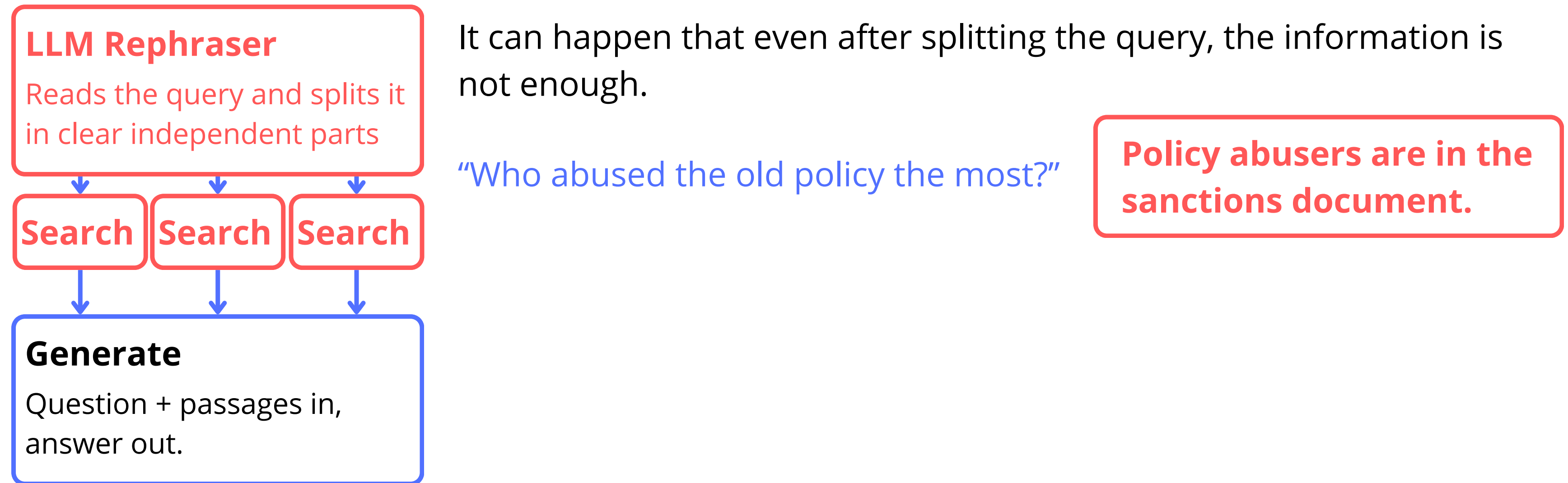
Reads the query and splits it
in clear independent parts

This is control flow, not content.

Rephrasing is one thing, but can we do more?

Why Agentic?

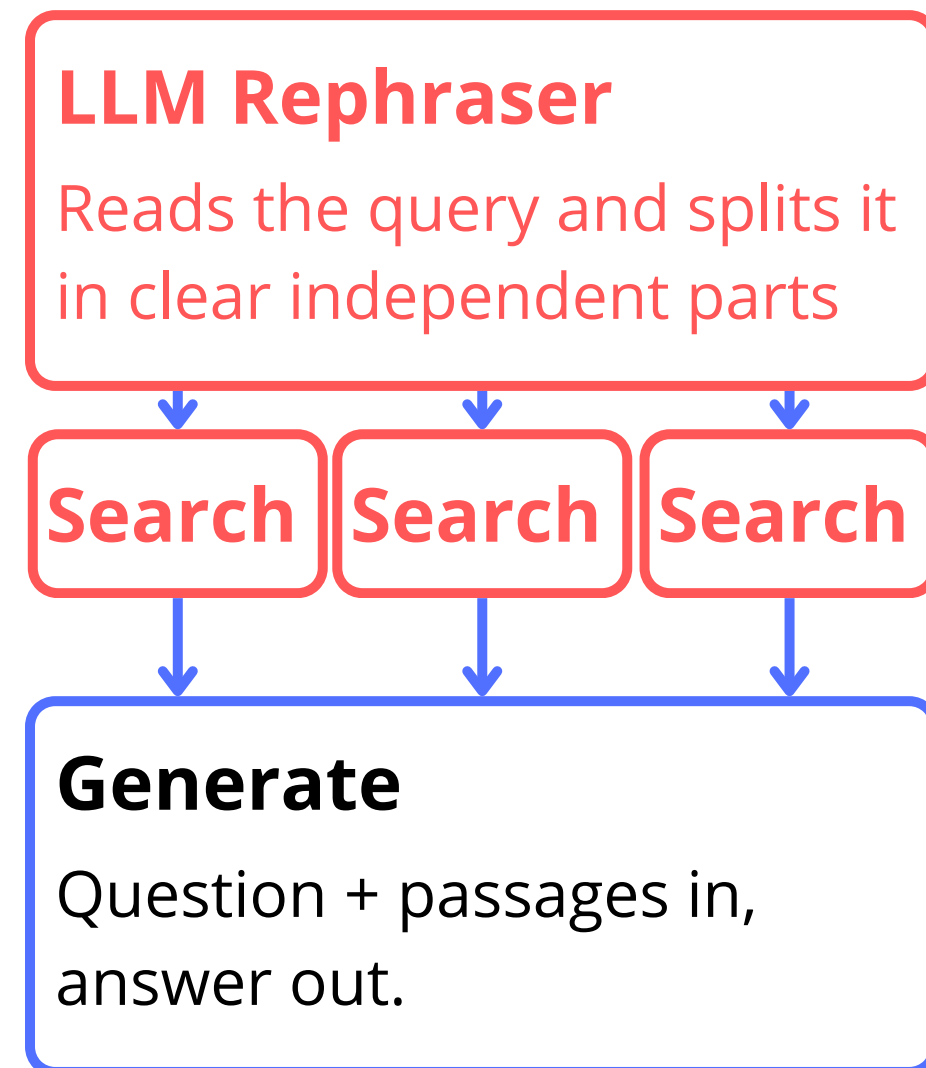
The LLM can take more decision, not only initial planning.



Search Pipeline, used at query time

Why Agentic?

The LLM can take more decision, not only initial planning.



Search Pipeline, used at query time

It can happen that even after splitting the query, the information is not enough.

“Who abused the old policy the most?”

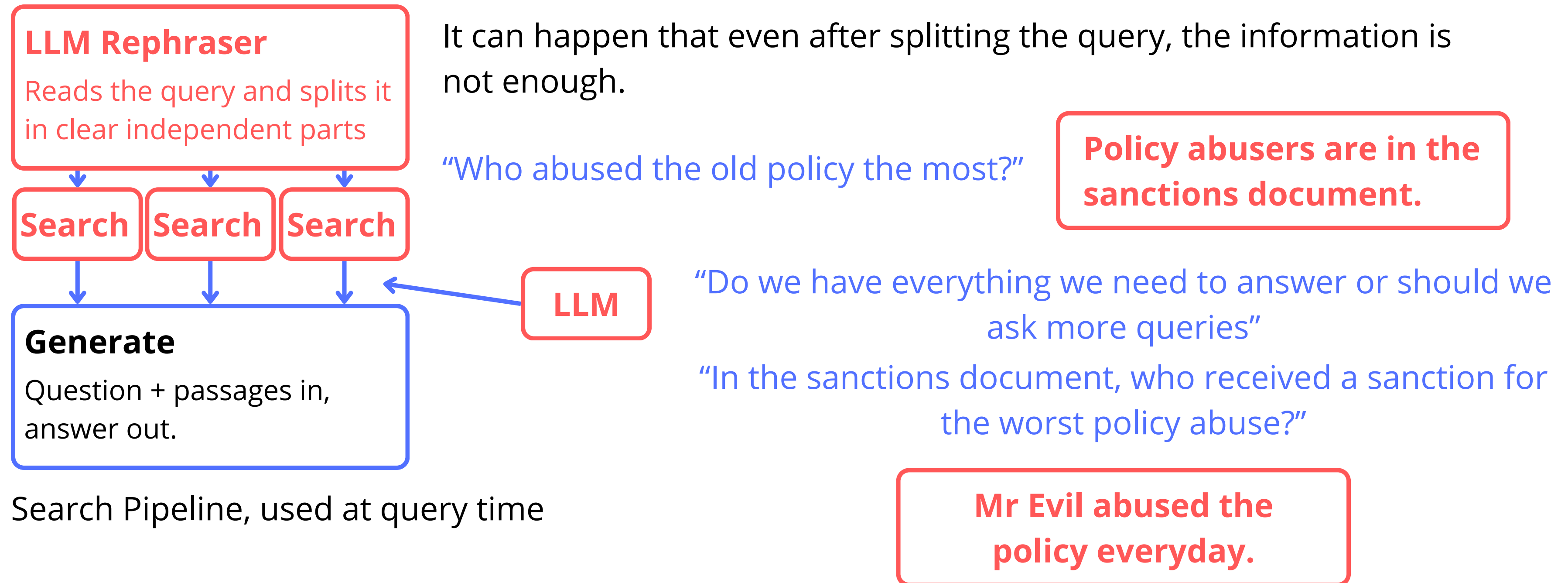
Policy abusers are in the sanctions document.

“In the sanctions document, who received a sanction for the worst policy abuse?”

Mr Evil abused the policy everyday.

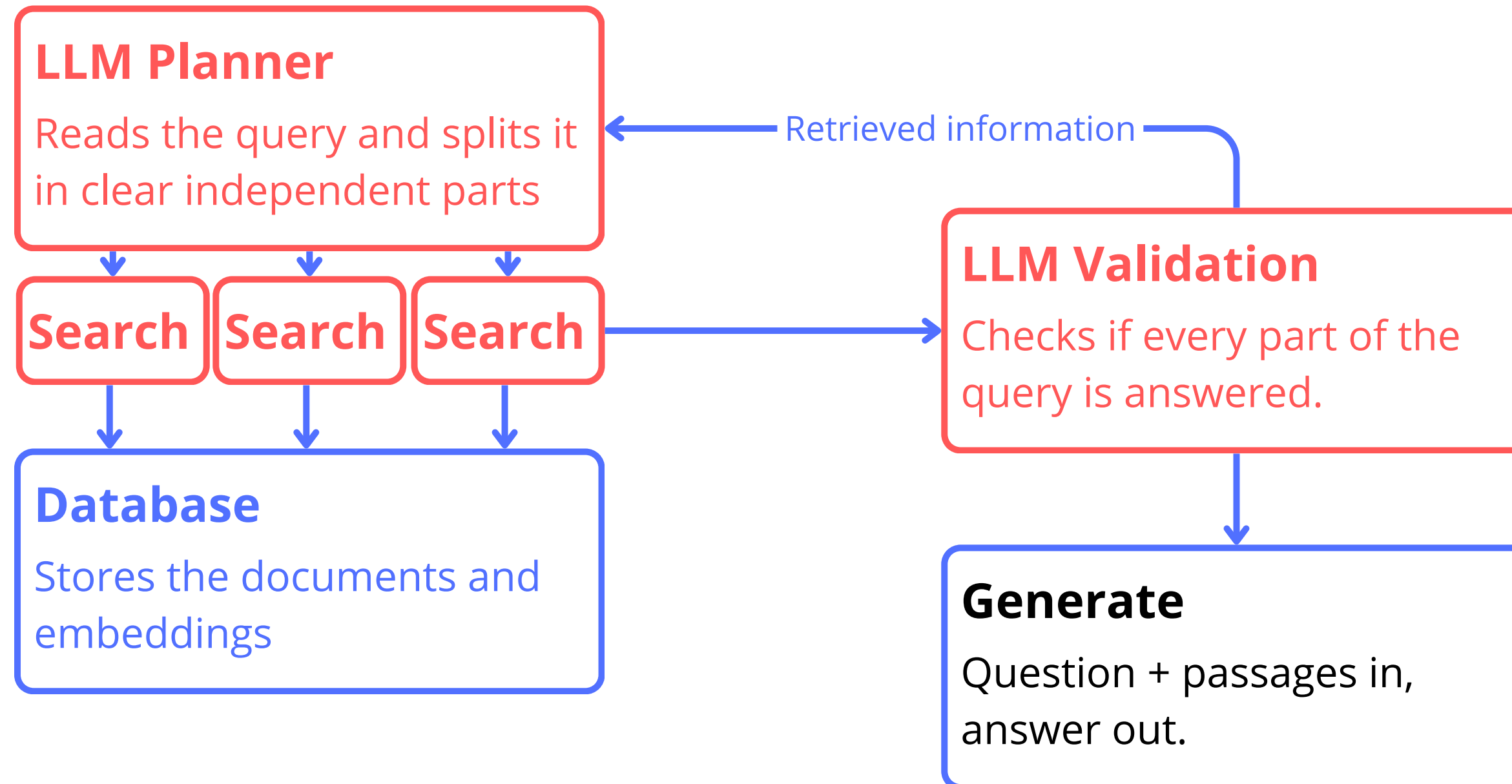
Why Agentic?

The LLM can take more decision, not only initial planning.



Agentic RAG V2

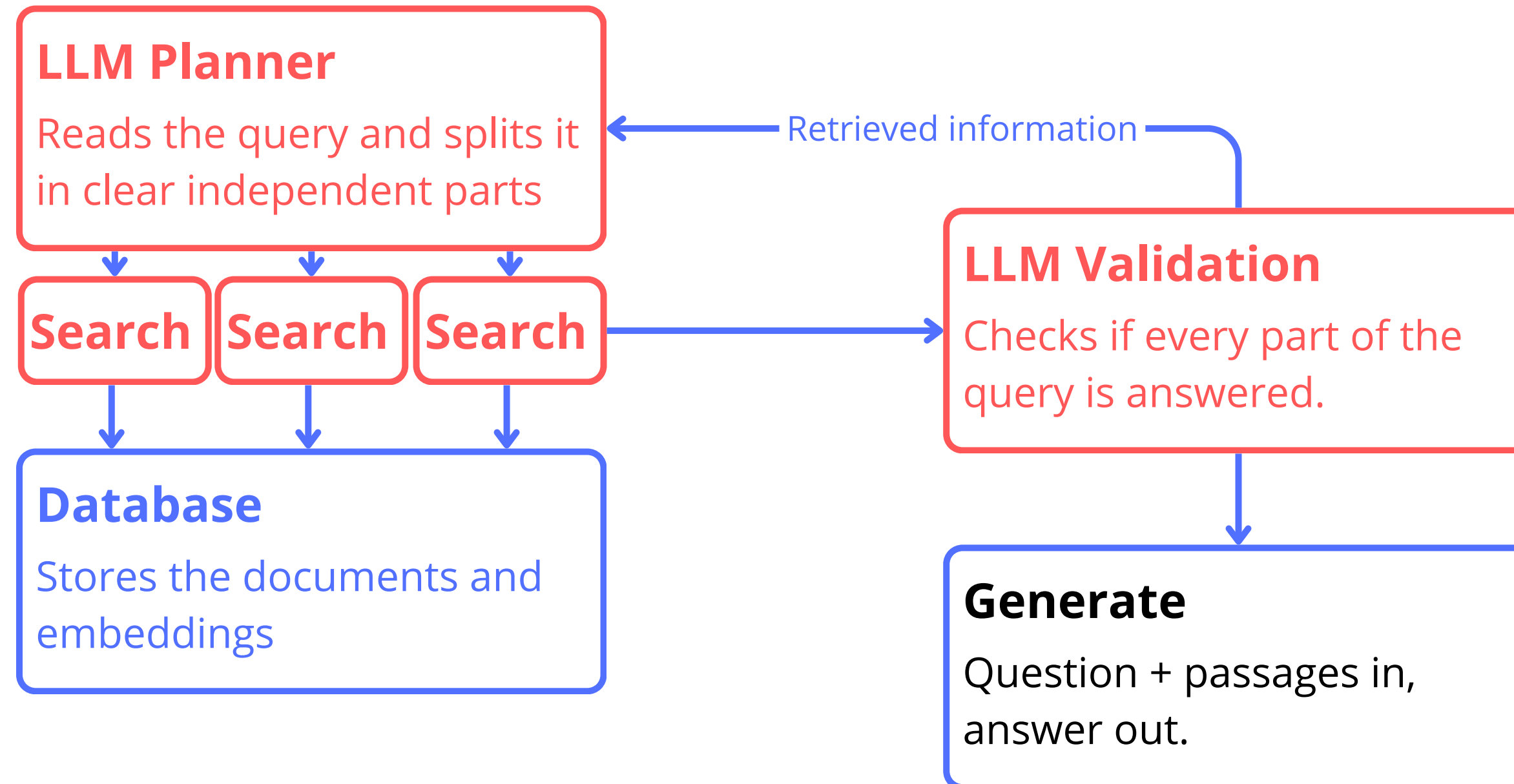
LLMs can manage the control flow for looking for solving hard questions.



The LLM judges if the evidence is enough for answering the question.

Agentic RAG V2

LLMs can manage the control flow for looking for solving hard questions.

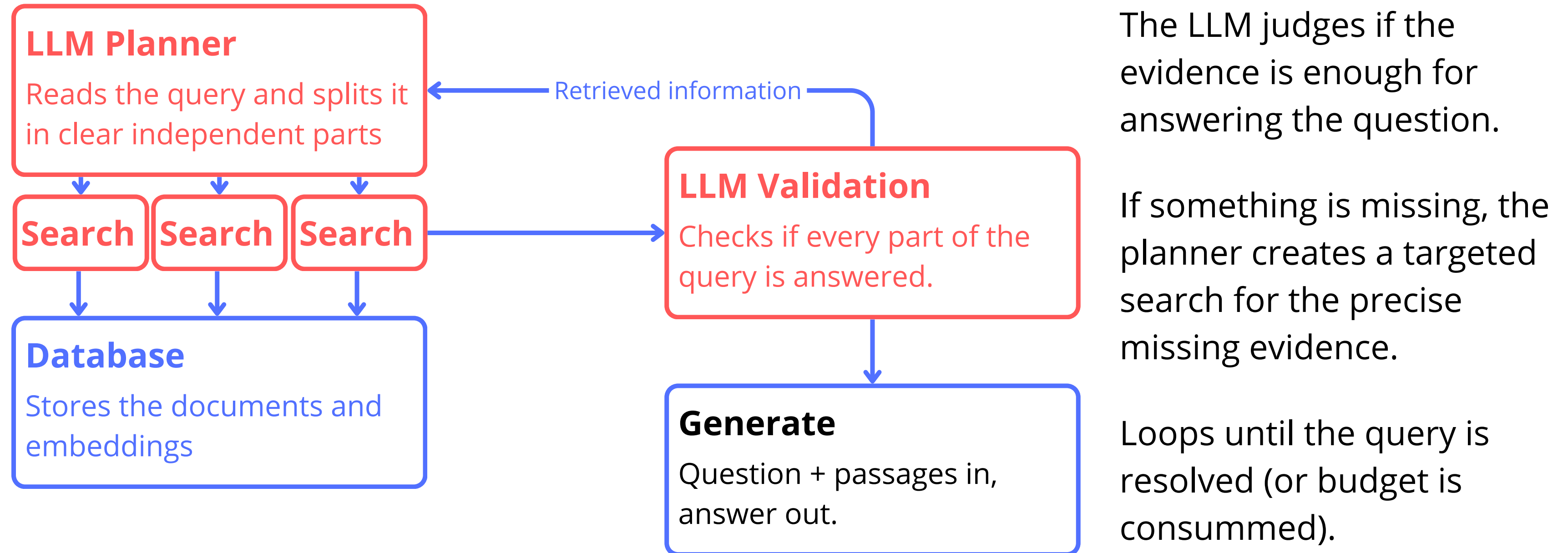


The LLM judges if the evidence is enough for answering the question.

If something is missing, the planner creates a targeted search for the precise missing evidence.

Agentic RAG V2

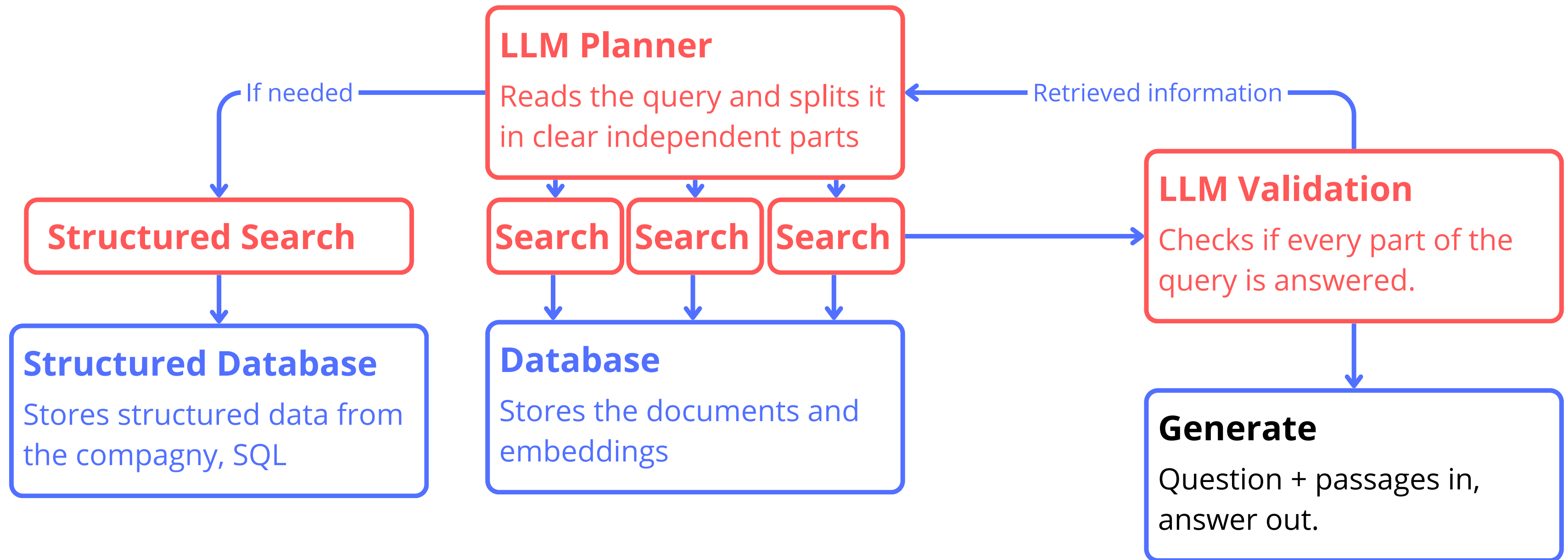
LLMs can manage the control flow for looking for solving hard questions.



The process and the search goals are defined by the LLM, based on the current needs.

Agentic RAG V3

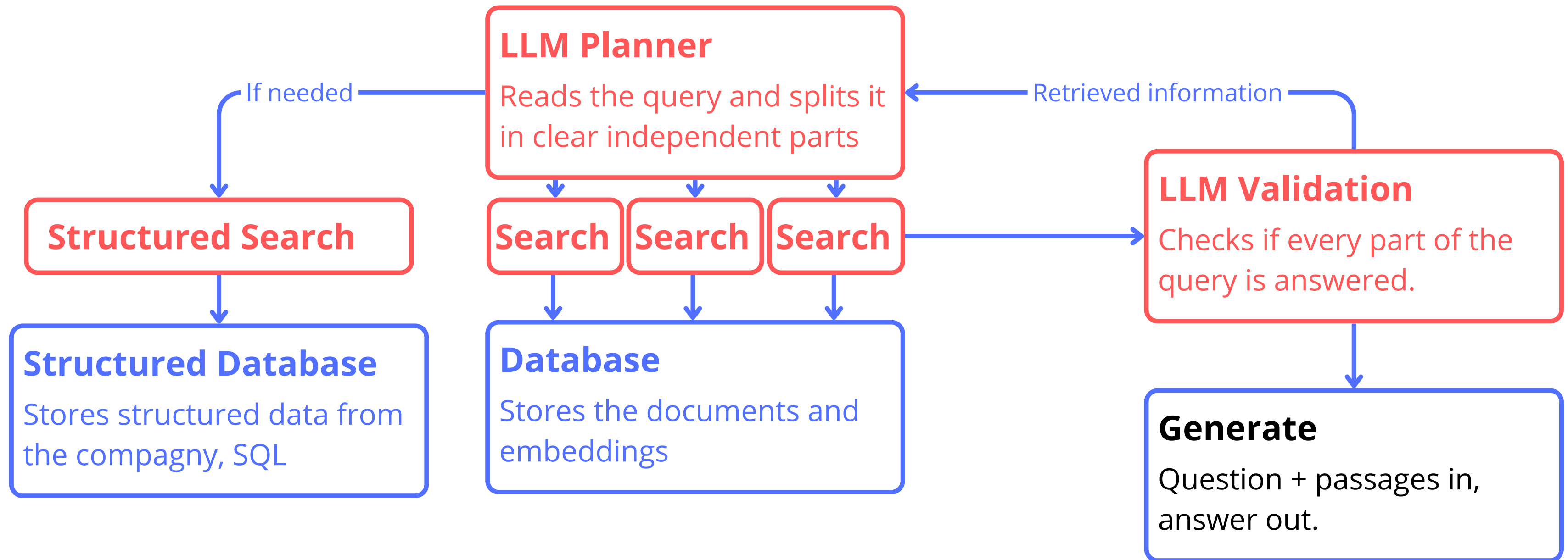
They can also spawn different type of searches and use tools.



Tools can allow to fetch information that is not in the documents but that might be useful to answer the question. More on this in the agentic Weekend.

Agentic RAG V3

They can also spawn different type of searches and use tools.



Tools can allow to fetch information that is not in the documents but that might be useful to answer the question. More on this in the agentic Weekend.

Graph RAG Motivations

“Who runs the team that owns the service behind last month's most critical outage?”

Answer is built from multiple intermediary documents.

Chunks are independent.

The index knows that two passages are similar, it has no idea they are about the same person.

Graph RAG Motivations

“Who runs the team that owns the service behind last month's most critical outage?”

Answer is built from multiple intermediary documents.

Chunks are independent.

The index knows that two passages are similar, it has no idea they are about the same person.

→ Multi Hop questions about the same entity can fail.

Last Month Outages → Most critical outages → Service Behind → Service Team → Team Leader

This is a **Local Query**, anchored around the entities that it is composed of.

Graph RAG Motivations

“Who runs the team that owns the service behind last month's most critical outage?”

Answer is built from multiple intermediary documents.

Chunks are independent.

The index knows that two passages are similar, it has no idea they are about the same person.

→ Multi Hop questions about the same entity can fail.

Last Month Outages → Most critical outages → Service Behind → Service Team → Team Leader

This is a **Local Query**, anchored around the entities that it is composed of.

“What other critical consequences were caused by the disconnection?”

The question is about the corpus, not a specific document.

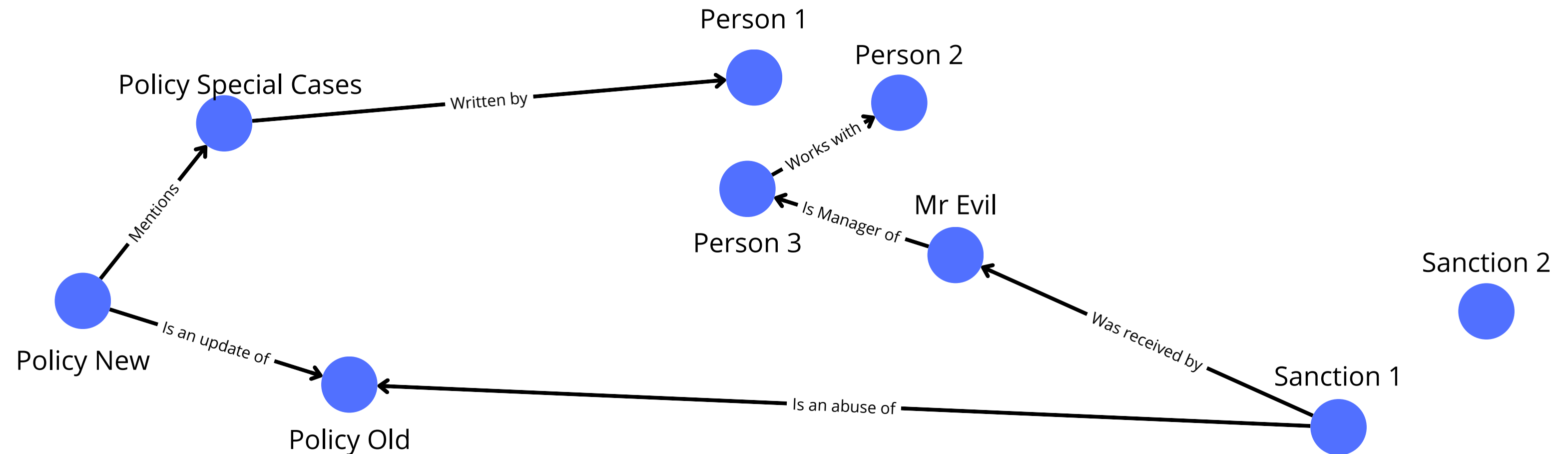
This is a **Global Queries**, no anchor, exploration of the corpus.

Graph RAG

Build a graph that connects the chunks.

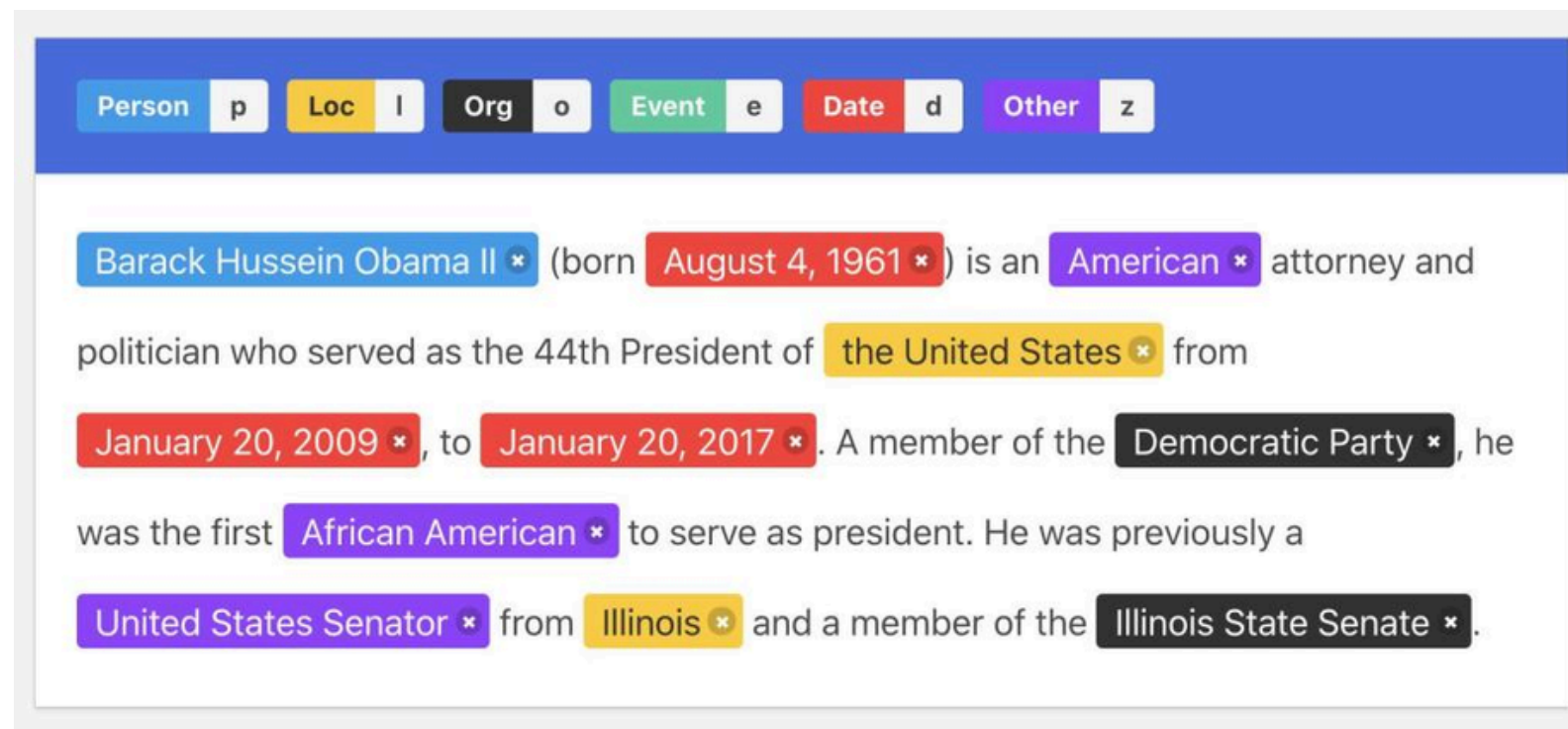
Nodes are entities, they represent people, events, policies, places...

Edges are relationships stated in the text, each one keeps a pointer back to the passage it came from.



Extract entities and relations

To build the graph you first have to understand what elements compose it.



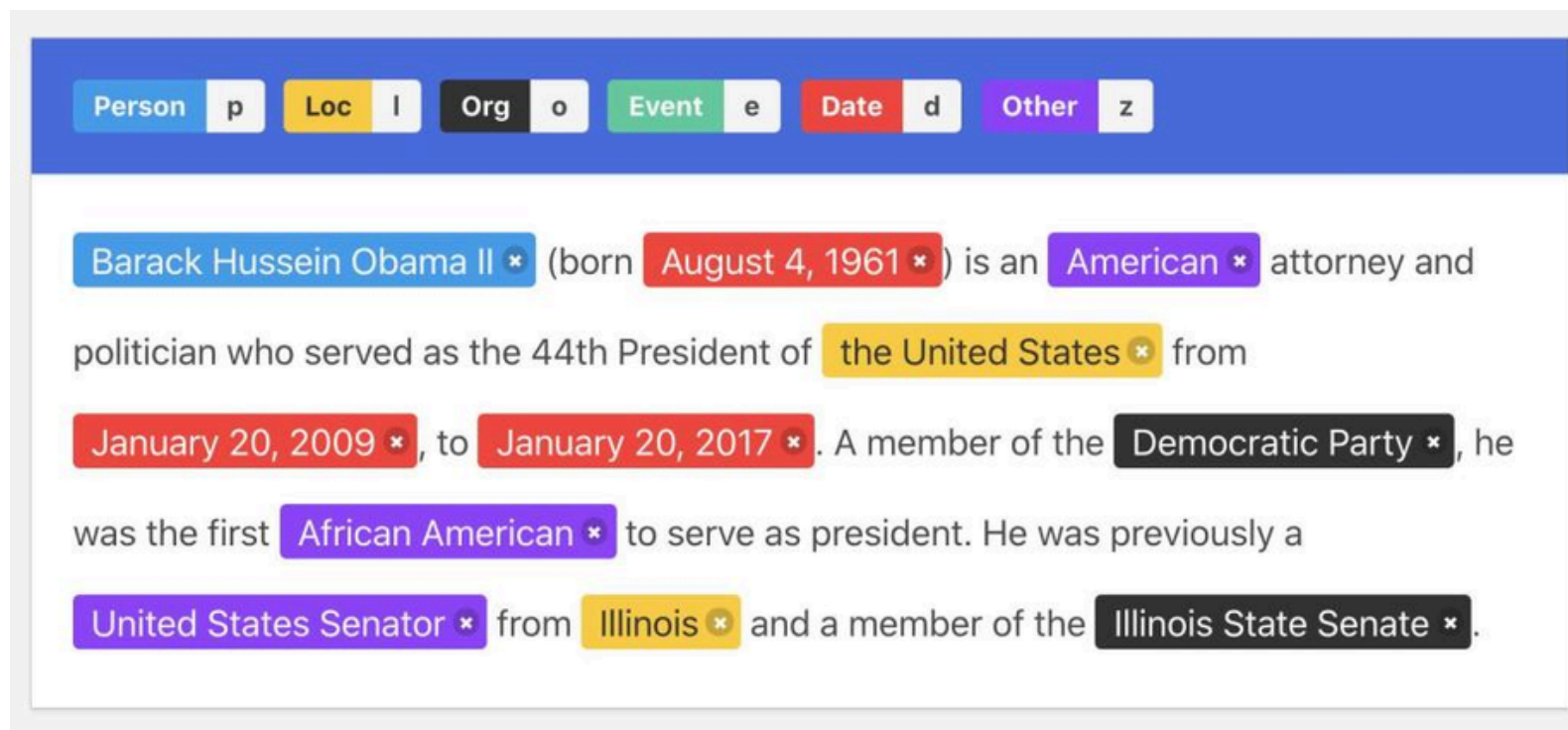
The screenshot shows a text annotation interface. At the top, there is a legend bar with six categories: Person (p, blue), Loc (l, yellow), Org (o, black), Event (e, green), Date (d, red), and Other (z, purple). Below the legend, a paragraph of text is displayed with several words highlighted in colored boxes corresponding to the legend categories. The text is: "Barack Hussein Obama II * (born August 4, 1961 *) is an American * attorney and politician who served as the 44th President of the United States * from January 20, 2009 *, to January 20, 2017 *. A member of the Democratic Party *, he was the first African American * to serve as president. He was previously a United States Senator * from Illinois * and a member of the Illinois State Senate *." The highlighted words are: Barack Hussein Obama II * (Person), August 4, 1961 * (Date), American * (Other), the United States * (Loc), January 20, 2009 * (Date), January 20, 2017 * (Date), Democratic Party * (Org), African American * (Other), United States Senator * (Other), Illinois * (Loc), and Illinois State Senate * (Org).

Your goal is to annotate the words in the text, recognizing the elements, that fall into some categories that you have defined.

These will be your nodes.

Extract entities and relations

To build the graph you first have to understand what elements compose it.



Your goal is to annotate the words in the text, recognizing the elements, that fall into some categories that you have defined.

These will be your nodes.

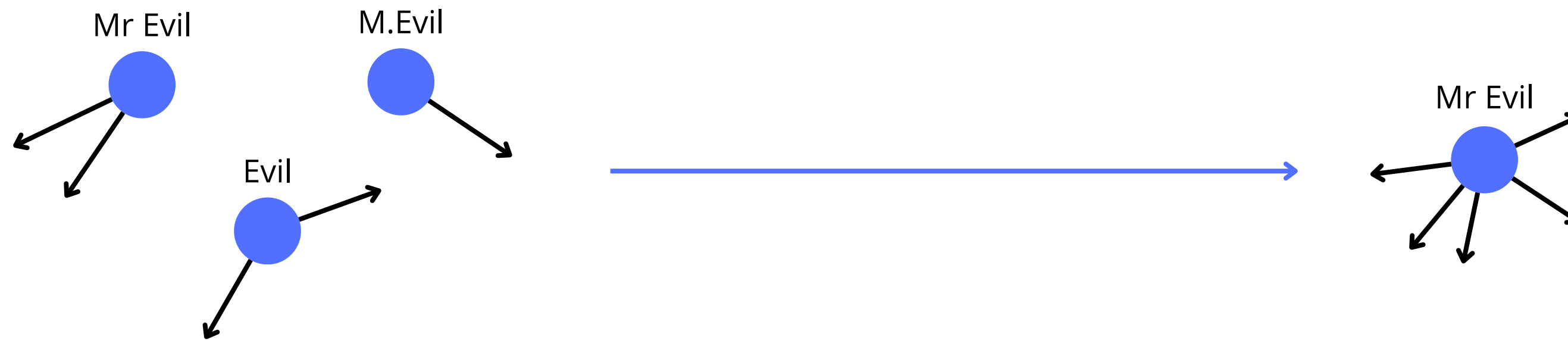
You then also derive the relationships between these elements, as triplets:
“Subject” → “Relation” → “Object”

This part is done offline, and is expensive, processing a lot of text.

It can be done with a model analyzing all the chunks over all the corpus.

Extract entities and relations

Entities and Relationship extraction needs a post processing.

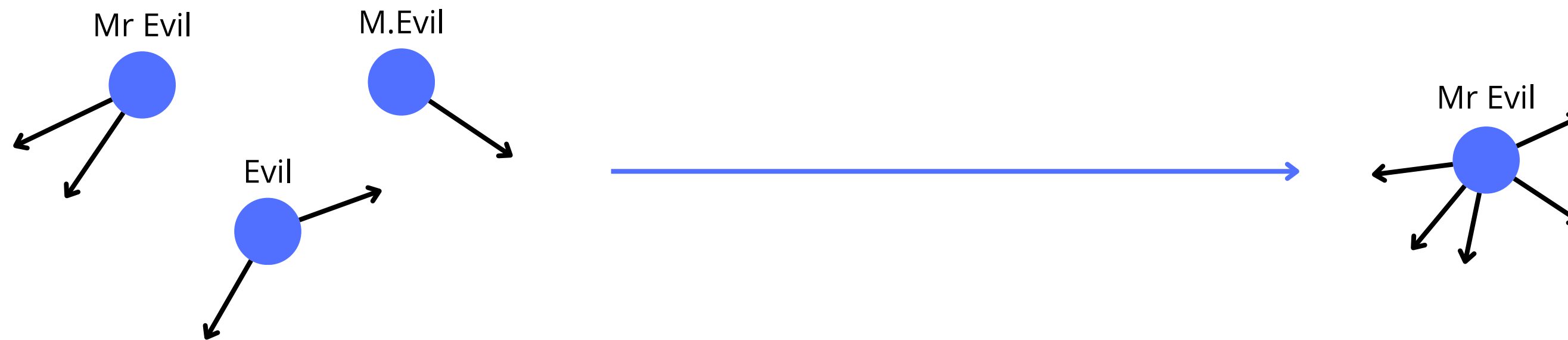


Apply a merge if the entities are “Similar”:

- Name similarity
- Entity Type
- Semantic Similarity of the Context
- Embedding Similarity

Extract entities and relations

Entities and Relationship extraction needs a post processing.



Apply a merge if the entities are “Similar”:

- Name similarity
- Entity Type
- Semantic Similarity of the Context
- Embedding Similarity

Over Merging → Two different entities become the same, their relations are shared.

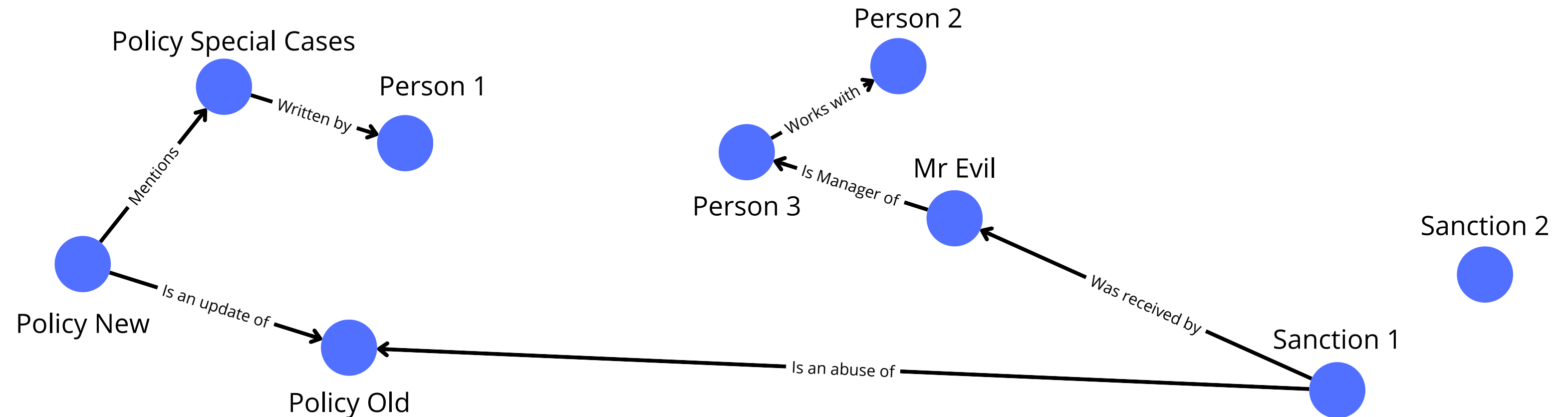
Under Merging → An entity is split in two, some relations might be missing in search.

Graph RAG

Once the graph is built, you want to build the communities

Some groups of nodes reference themselves way

Edges are relationships stated in the text, each one keeps a pointer back to the passage it came from.

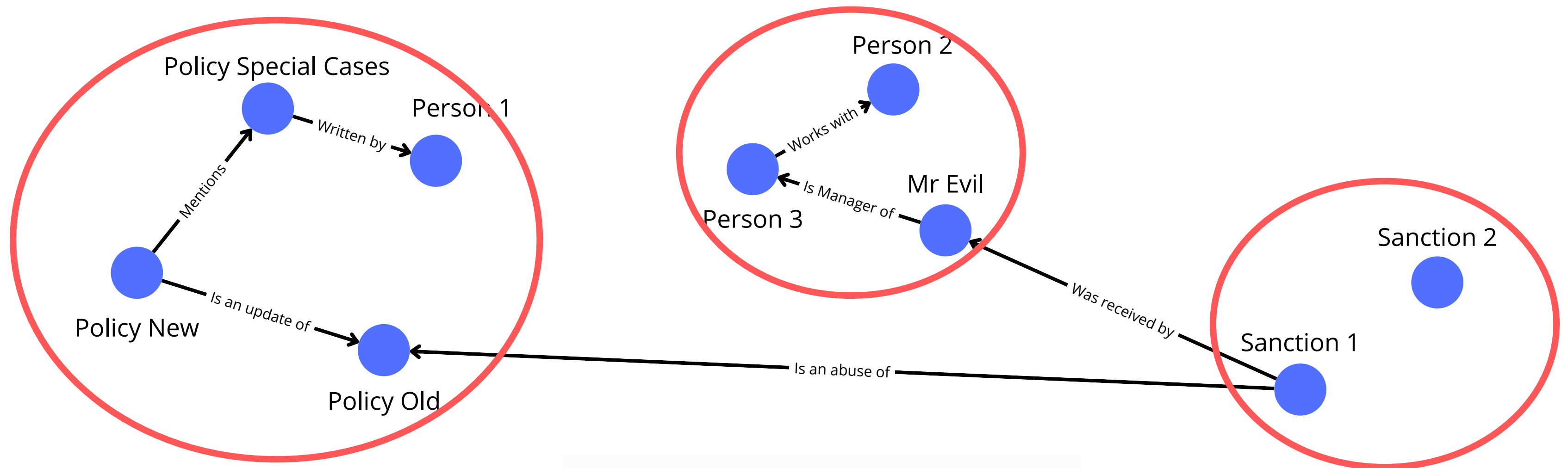


Graph RAG

Once the graph is built, you want to build the communities

Some groups of nodes reference themselves way

Edges are relationships stated in the text, each one keeps a pointer back to the passage it came from.



We do this by computing the

$$\text{good community} \approx \frac{\text{internal connectivity}}{\text{external connectivity}}$$

Graph RAG

A way to compute this is the Louvain Algorithm.

$$Q = \frac{1}{2m} \sum_{ij} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j)$$



$$A_{ij} = \begin{cases} 1 & \text{if } i \text{ and } j \text{ are connected} \\ 0 & \text{otherwise} \end{cases}$$

“Does this proposed grouping contain more connections inside its communities than we would expect by chance?”

Graph RAG

A way to compute this is the Louvain Algorithm.

$$Q = \frac{1}{2m} \sum_{ij} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j)$$

$$A_{ij} = \begin{cases} 1 & \text{if } i \text{ and } j \text{ are connected} \\ 0 & \text{otherwise} \end{cases}$$

“Does this proposed grouping contain more connections inside its communities than we would expect by chance?”

K_i is the degree of the node

2m is the total number of edges

→ The chances that these two nodes would be connected, based on their degree if the graph was random

Graph RAG

A way to compute this is the Louvain Algorithm.

$$Q = \frac{1}{2m} \sum_{ij} \left(A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j)$$

“Does this proposed grouping contain more connections inside its communities than we would expect by chance?”

$$A_{ij} = \begin{cases} 1 & \text{if } i \text{ and } j \text{ are connected} \\ 0 & \text{otherwise} \end{cases}$$

k_i is the degree of the node

$2m$ is the total number of edges

→ The chances that these two nodes would be connected, based on their degree if the graph was random

A high score is induced by two nodes being connected, while a random graph wouldn't have connected them with high probability

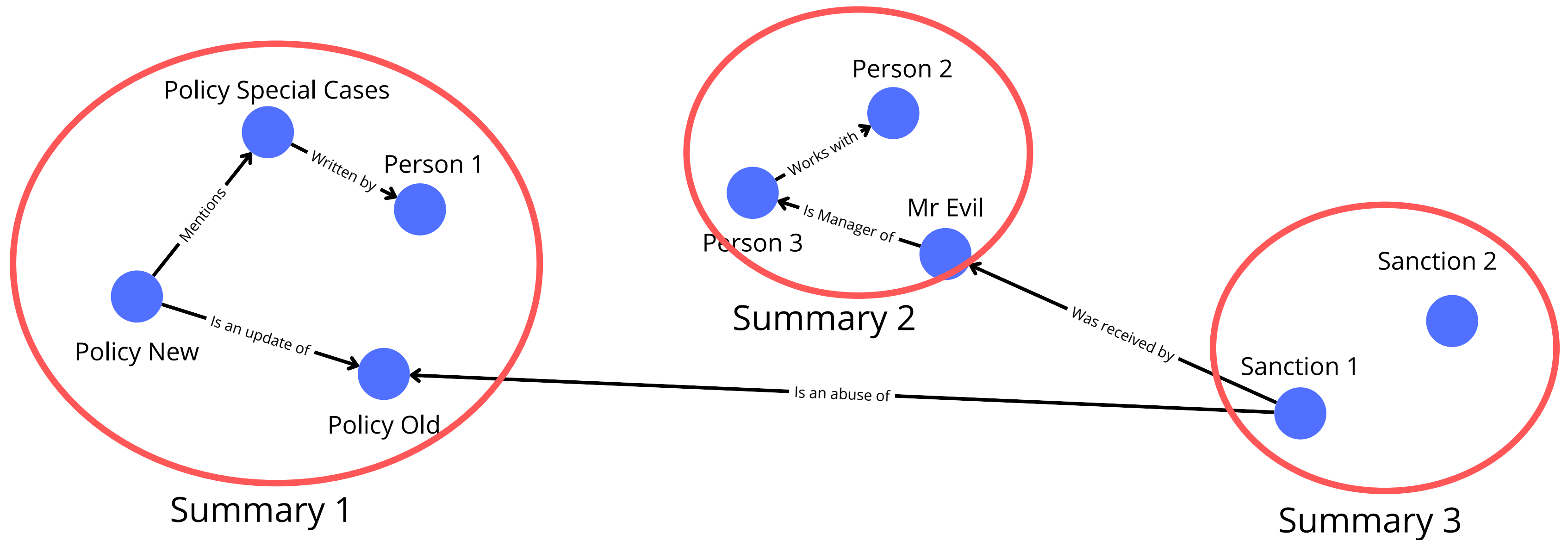
→ Strong evidence of a relationship

Try multiple assignments and take the one with the best score.

Graph RAG

You can then use the nodes, relationships and chunks from a community to summarize it.

This way you have some text to compare to, that would give information about the whole group.

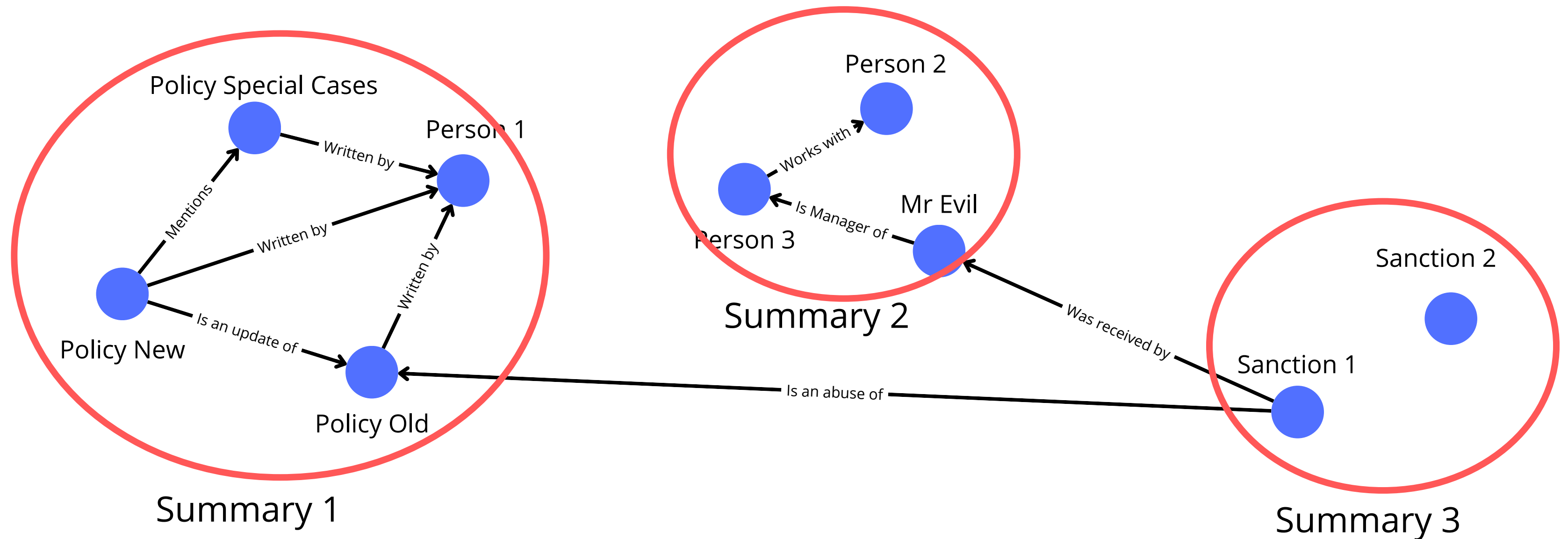


Local Search

Some searches are anchored on one node, and look into the neighborhood.

“Which is the last policy written by Person 1?”

The search starts at the Person 1 node and examines the neighbouring nodes.



The search expands to the multi hop neighbors until the information has been found.

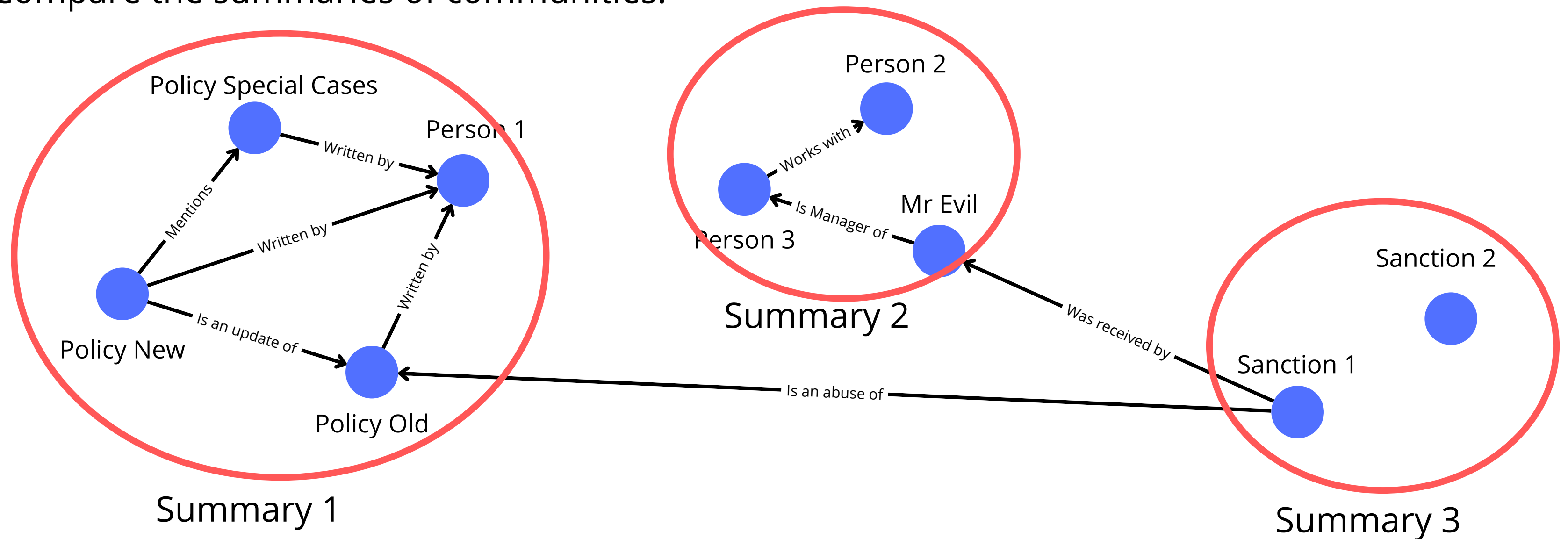
Global Search

Some searches are more general, no specific entity is specified.

“Which were the most critical sanctions?”

There is no specific start point, the answer could be anywhere in the corpus.

→ Compare the summaries of communities.



Then take the content of the relevant communities as evidence, or go more precisely.

Graph RAG

Can help us answer multi hop, complex questions

The graph contains a lot of value that simplifies the searches for questions, while not losing information.

The Local and Global searches give two ways of answering precise and vague questions.

Global Search on a Local question → The meaning might be diluted in the summary

Local Search on a Global question → The search might miss relevant chunks

Graph RAG

Can help us answer multi hop, complex questions

The graph contains a lot of value that simplifies the searches for questions, while not losing information.

The Local and Global searches give two ways of answering precise and vague questions.

Global Search on a Local question → The meaning might be diluted in the summary

Local Search on a Global question → The search might miss relevant chunks

Building the Graph is very expensive, but is offline. Once built, you can answer as many questions as needed.

Adding a document means:

- Extracting the edges and entities
- Recomputing the Communities
- Recomputing the summaries

When is Graph not worth it?

Questions are **single-hop**. If the answer is reliably in one passage, you are paying for structure nobody traverses.

The corpus is small. Under a few hundred documents, a long-context model can read a good chunk of it directly.

The text has few entities. Reference manuals, code, log dumps extraction produces a thin, uninteresting graph.

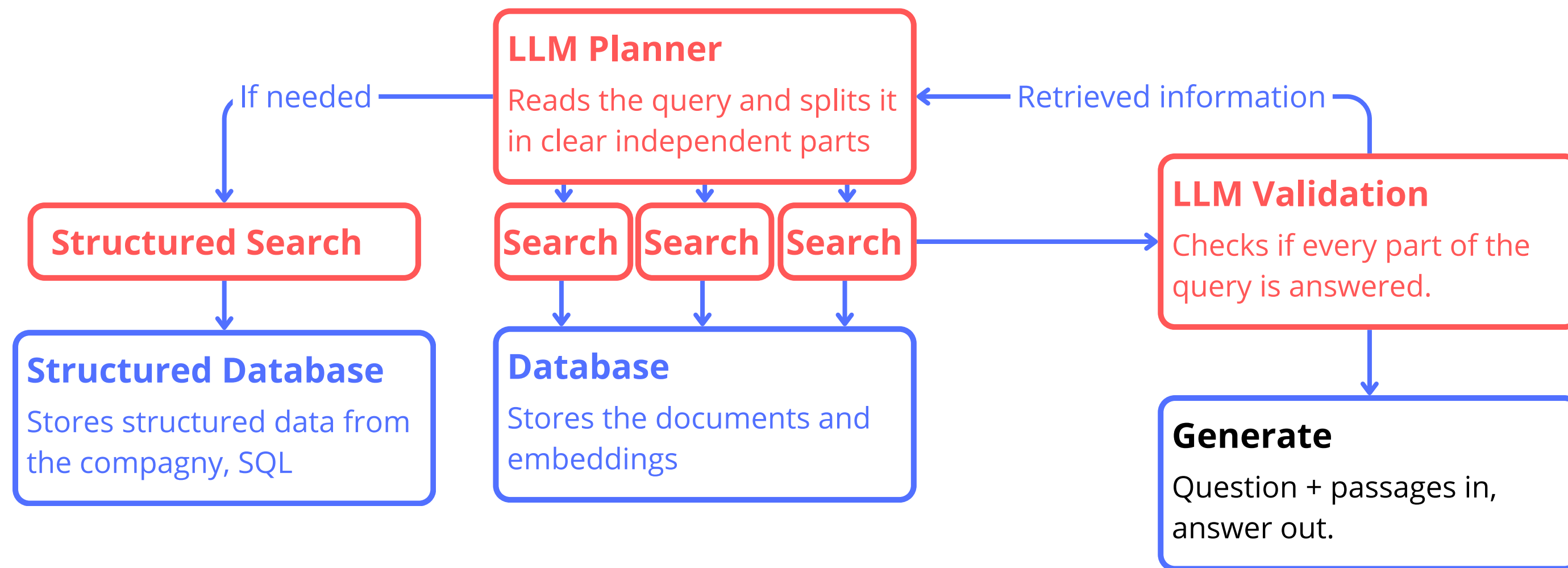
The data already has structure. If your facts live in a database, query the database. Do not extract a graph from prose about it.

It changes by the minute. Re-partitioning and re-summarising cannot keep up with a corpus that turns over hourly.

Hybrid Search

The Graph enables Search to better understand multi-hop questions and more complex relationships between chunks. Agentic RAG enables thinking about vague questions and validating the progress of the search.

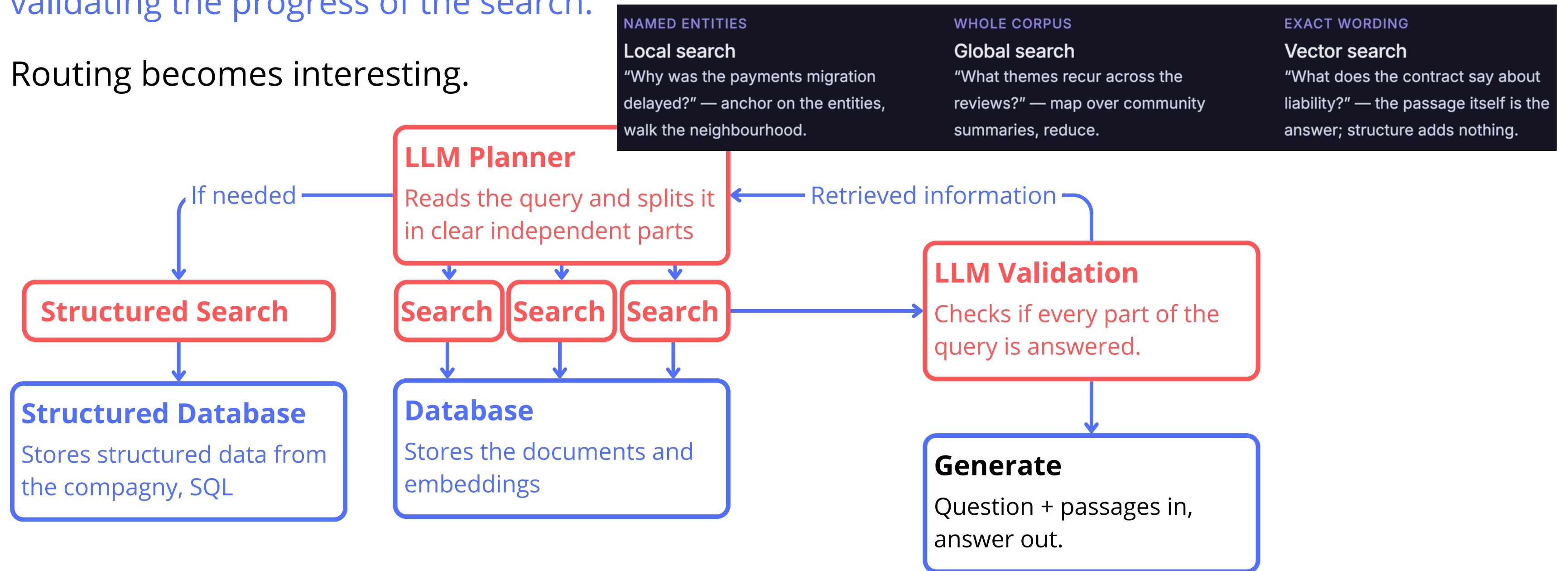
Routing becomes interesting.



Hybrid Search

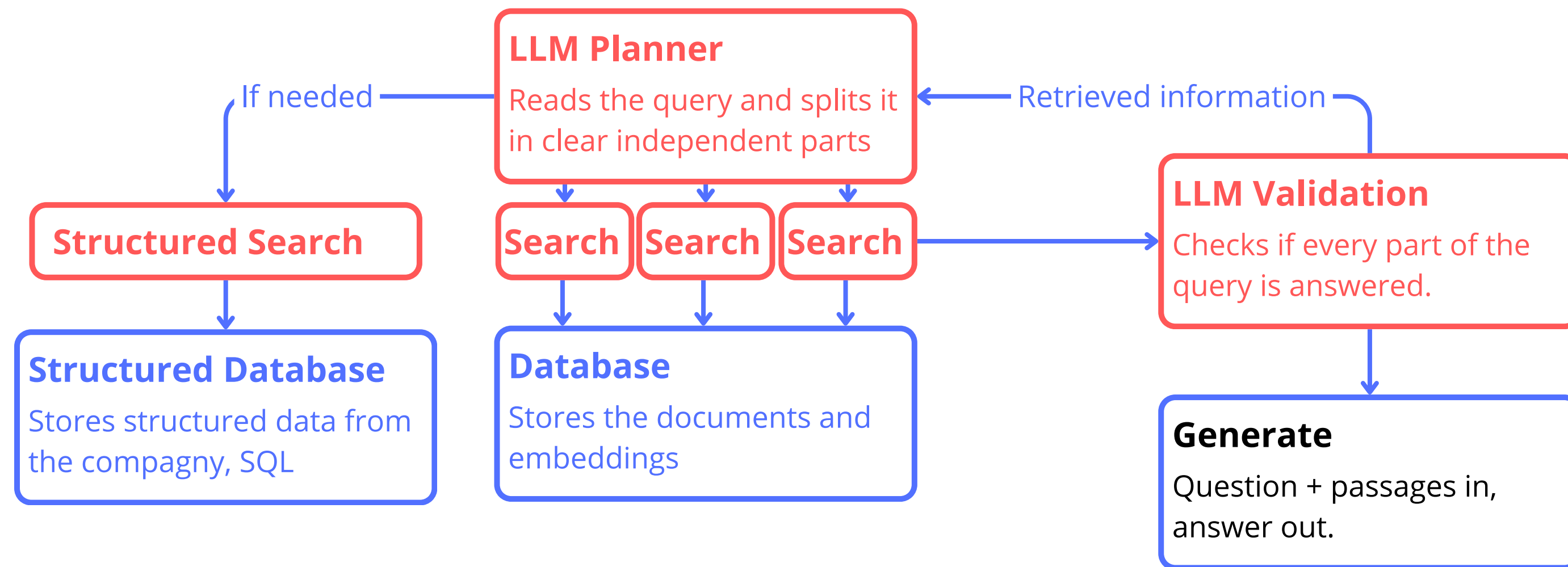
The Graph enables Search to better understand multi-hop questions and more complex relationships between chunks. Agentic RAG enables thinking about vague questions and validating the progress of the search.

Routing becomes interesting.



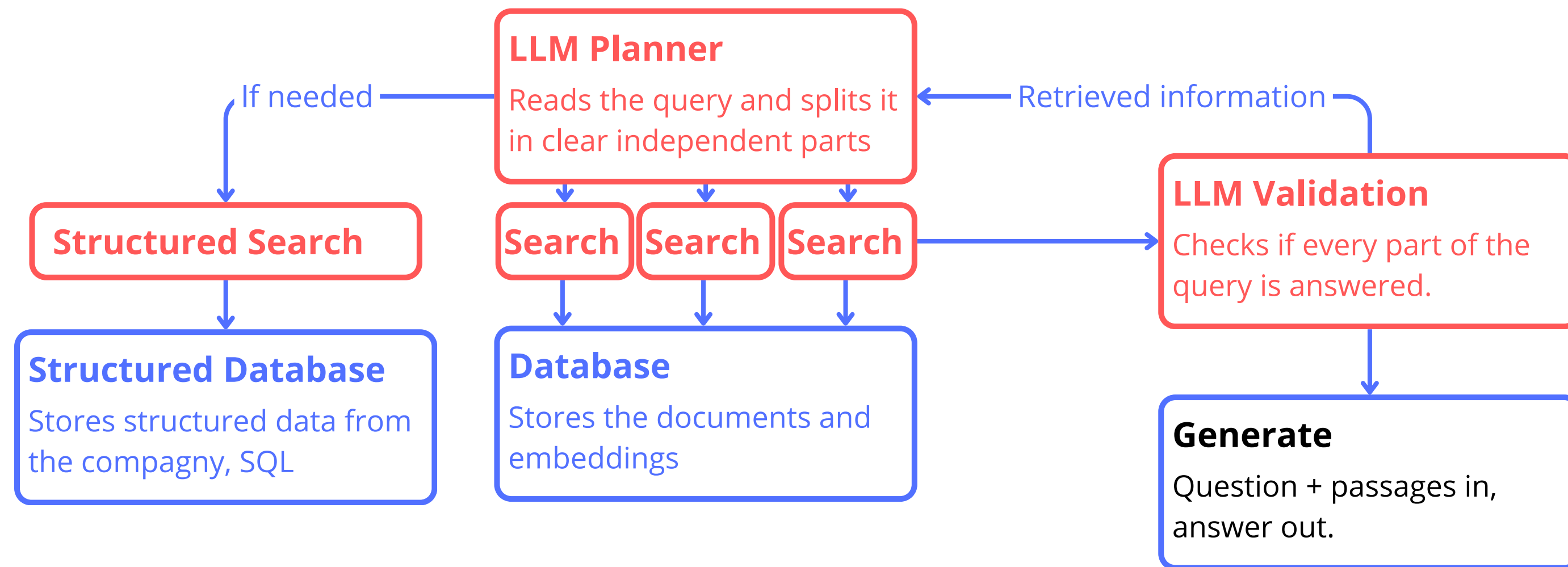
Hybrid Search

“Compare how we handled the last two payment outages, and say what we changed.”



Hybrid Search

“Compare how we handled the last two payment outages, and say what we changed.”
Planning and routing: both parts name entities, so local search on the graph, in parallel.

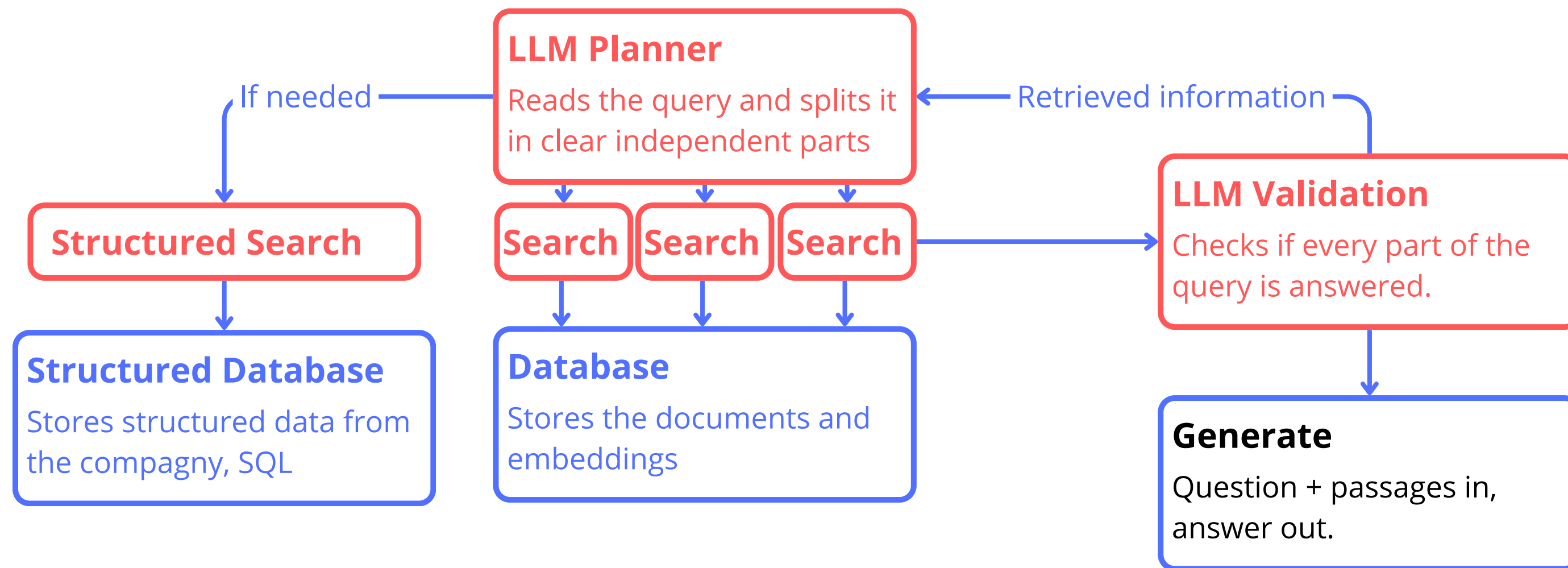


Hybrid Search

“Compare how we handled the last two payment outages, and say what we changed.”

Planning and routing: both parts name entities, so local search on the graph, in parallel.

Search: start on each outage node, expand to teams, services and postmortems, pull the source passages behind those edges.



Summary

- Plain RAG fails in two distinct ways: it cannot decide anything, and it cannot see connections
 - Agentic RAG puts a model in the control flow: validate, rewrite, decompose, route, grade, retry, verify. It pays for it in calls and latency.
 - Graph RAG pays up front to turn text into entities, relations and community summaries, and gets multi-hop and corpus-wide questions in return.
 - Both add moving parts. Neither is a default: the question shapes tell you whether you need them.
-
- Start with a plain RAG, and note the queries on which it fails
 - If your queries fail because they are too complex, or vague, implement Agentic Control Flow
 - If your queries fail because the reasonings are multi steps, or relations between entities are important, add a Graph RAG.